Parallel and Distributed Systems: One Shot Project

Matthew Bernardi
m.bernardi18@studenti.unipi.it

September 1, 2026

Problem Given a sparse matrix $A \in R^{N \times N}$ in the CSR form and an initial vector $\mathbf{x}_0 \in R^N$, iteratively compute the sparse matrix-vector operation followed by a normalization step, obtaining $\mathbf{y}_0 \in R^N$. Then, set $\mathbf{x}_1 = \mathbf{y}_0$. Every `EPOCH_LEN` epochs, shift the rows of the A matrix.

In the following, we'll take a closer look at the different implementations of the algorithm: Thread, OpenMP, MPI+OpenMP.

1 Implementation strategies

The main algorithm consists in the following steps:

1. Generation of the A matrix,
2. Generation of $\mathbf{x}$,
3. Initial normalization of $\mathbf{x}$,
4. Iteratively repeat:
   (a) Possible row shift (depending on the current iteration and the epoch length),
   (b) SpMV function call,
   (c) Normalizing step,
   (d) Swap between $\mathbf{x}$ and $\mathbf{y}$.
5. SpMV function call,
6. Computation of the Rayleigh value using the `dot` function,
7. Computation of the checksum value.

1.1 Thread version

In the Thread version, I implemented a ThreadPool in which a fixed number of threads waits for tasks to execute. The number of workers is defined in the command line arguments via the `-t` parameter. The choice of using a ThreadPool instead of spawning a new thread for each task is dictated from the potentially big number of tasks that are generated during the program execution. Assuming the same value for the `-chunk-size` parameter from command line, the number of tasks grows with the dimension N of the A matrix and $\mathbf{x}$ vector. Using a fixed number of threads in a thread pool lowers significantly the overhead to spawn new threads, since it only spawns `-t` threads. The thread pool has a `barrier()` method used to wait for the completion of all the tasks in the task queue, and it's used at the end of each normalization step, of the SpMV function call, of the Rayleigh value computation and checksum computation. The generation of $\mathbf{x}$ is done sequentially for simplicity (otherwise each thread should have its own `rng` object).

For the `dot` and `checksum_vector` functions, each thread computes the resulting value of the data it has to work on in a partial result, and then a reduction step is taken to get the single resulting value.

It is possible to run the program specifying a scheduling technique using the `-scheduling` parameter, which accepts only the values `static`, when we want to equally divide the workload among the threads, or `dynamic`, when we want to manually specify the size of the chunks to work on using the `-chunk-size` parameter. The dynamic scheduling implements a *work-stealing* paradigm: define the tasks as chunks of data we work on, and each worker, whenever he can, steals a task from the queue, incrementing the `next_chunk` atomic variable, which indicates the index of the next chunk to compute.

Adding the `-chunk-size` parameter introduces a trade-off: smaller values provide fine grained load balancing, useful in case of irregular workloads, but increases the number of tasks created and hence the scheduling overhead. Large chunks reduce scheduling overhead but may increase the load imbalance.

1.2 OpenMP version

The OpenMP version of the SpMV program has the same concept of the dynamic scheduling in the Thread version: create a pool of workers waiting for a job, divide the workload into chunks of data, assign dynamically the tasks to the threads. The main structure of the OpenMP program is the following:

```
1   #pragma omp parallel
2   {
3       #pragma omp single
4       {
5           ...
6           for each iteration {
7               ...
8               for each chunk of data {
9                   #pragma omp task
10                  {
11                      ...
12                  }
13              }
14              #pragma omp taskwait
15              ...
16              // repeat the same pattern for following steps (normalization,
    dot and checksum)
17          }
18      }
19  }
```

where:

- **`#pragma omp parallel`**: defines an OpenMP parallel section, in which `num_threads` threads are created, and each one has the variables `x, y, partial_checksums, row_shift, rayleigh, checksum, num_chunks` shared. The threads (workers) wait for tasks to execute,
- **`#pragma omp single`**: section which is executed only by a single thread,
- **`#pragma omp task`**: creates a task to add to the queue,
- **`#pragma omp taskwait`**: works as a barrier, and waits until all the tasks generated in the context are completed.

Whenever a new task is generate in the `#pragma omp task` section, the runtime assigns the task to one of the waiting threads.

For the task-based OpenMP implementation, the chunk size directly determines the task granularity. With a problem size N and chunk size C, approximately

$$N_{\text{tasks}} = \left\lceil \frac{N}{C} \right\rceil$$

tasks are generated for each operation. Therefore, decreasing C increases the number of tasks and provides finer-grained scheduling, but also increases task creation and runtime management overhead. Increasing C has the opposite effect: fewer tasks are generated, reducing overhead but potentially worsening load balancing for irregular rows.

1.3 Task-Based vs Work-Sharing OpenMP

The alternative **work-sharing** version removes explicit tasks. All threads directly cooperate on the same loops through `#pragma omp for` constructs within a single `#pragma omp parallel` region. Loop-iteration distribution is therefore delegated to the OpenMP runtime through the `schedule` clause, rather than being manually implemented through `start`, `end`, and the number of chunks.

The SpMV kernel uses `schedule(dynamic, chunk_size)` because of the irregular matrix. With dynamic scheduling, when a thread finishes its current chunk, it receives another available chunk of rows, just like the dynamic scheduling in the Thread version. This reduces load imbalance caused by denser rows. The `chunk_size` parameter remains a trade-off between scheduling overhead and load-balancing quality.

The `dot` and `normalization` functions and checksum computation, have an approximately uniform cost per element, so they use static scheduling. Since the cluster OpenMP implementation does not support the `reduction` clause, the dot product and checksum are implemented through explicit per-thread partial values. Padding is added to avoid false sharing between accumulators. After an explicit barrier, a `#pragma omp single` block combines the partial values sequentially.

The `nowait` clause is used on the loops that compute per-thread partial values since it removes the implicit barrier at the end of the `omp for`. An explicit `#pragma omp barrier` is then used before the `single` block performs the final combination.

I kept the sequential operations inside the `#pragma omp single` block, just like in the task based version of OpenMP.

The only difference between the task based and work sharing versions is the work distribution strategy, but the algorithm remains the same. The task-based version offers greater flexibility and explicit control over task granularity, whereas the work-sharing version is simpler and avoids task creation overhead. Its performance depends on the number of threads, chunk size, and irregularity of the matrix.

1.4 MPI+OMP version

In the MPI+OMP version, the A matrix is partitioned among the different MPI ranks, and each partition is then processed in a parallel way using OpenMP. MPI is initialized with the `MPI_THREAD_FUNNELED` support level since only the master thread which manages the OpenMP's parallel region makes MPI calls. Therefore, the `#pragma omp single` is replaced with `#pragma omp master`.

A simple implementation of the MPI partitioning is distributing the number of rows of the A matrix across the P MPI ranks, assigning almost N/P rows to each rank. This could lead to an unbalanced workload for some nodes in case of irregular matrix generation. An optimal implementation assigns n/P elements of A (where n is the number of non zero elements) to each partition, ensuring work balancing. This approach is obviously harder to deal with in the

implementation, since there would be multiple ranks with elements from the same row, and the row-column product would be more difficult to implement correctly.

My solution sits in the middle: I decided to assign consecutively the rows, starting from the first, to the current rank, until adding another row would move the number of assigned non zero elements further from the target (n/P), then assign the next rows to the next rank with the same policy. The last rank takes the remaining rows. This strategy does not always yield the best partitioning, as we may end up assigning the majority of the non zero elements to the last rank. It represents a good compromise between assigning contiguous rows (and not singular elements) to ranks and balancing the workload.

After rank 0 has determined the row partitioning, it produces the `row_counts` and `row_displacements` vectors, and sends them to all the other ranks via `MPI_Bcast` (rank 0 needs to have all the partitioning information). Then rank 0 partitions the A matrix in CSR form to all the other ranks using `MPI_Scatterv`. Then the real SpMV algorithm with OpenMP is executed. Each rank produces a local output of the SpMV applied to the rows of the matrix A it received, and the results are gathered with `MPI_Allgatherv` (all the ranks need the vector in the next iteration), then the resulting vector $\mathbf{y}$ is rotated using the `row_shift` parameter. The idea behind it is that the following equation holds:

$$(PA)\mathbf{x} = P(A\mathbf{x})$$

so, applying the rotation to $\mathbf{y} = A\mathbf{x}$ is equivalent to applying the rotation to A and then multiply it with $\mathbf{x}$. This transformation changes the communication/computation trade-off: physical redistribution is avoided, but an $O(N)$ vector permutation is performed at every iteration. Using this implementation, the epoch transition time is negligible (since it's just the cost of the logical shift represented by the `row_shift` parameter), since each rank keeps the same portion of the A matrix for the entire program life. The price to pay is a total of $O(N)$ for each iteration (locally) to permute the $\mathbf{y}$ vector, in order to avoid the redistribution of the rows of the A matrix across ranks. This is an implementation choice, guided by the fact that rank-to-rank communication is often much more expensive than the local computation, even if the communication happens each `EPOCH_LEN` epochs, and the computation happens each epoch.

2 Correctness verification

The correctness verification has been done using two kind of validations:

- **Strong verification**: done through result vector dump on a file,
- **Weak verification**: check that the error between the rayleigh value obtained through the parallel version (Thread, OpenMP and MPI+OMP) and the one obtained through the sequential version falls within an acceptance range, called the `tolerance` hyperparameter, with a default value of 10^{-9}, in order to provide strict numerical similarity between the two solutions but still accept small numerical differences caused by the order of floating-point operations.

Weak verification is done using a tolerance range, checked during the execution of the experiments, because the rayleigh value in parallel implementations cannot be bitwise identical to the sequential version's one because of the operation reordering done with the parallelization.

The checksum is used as an additional diagnostic value rather than as the primary floating-point correctness criterion. Since the checksum depends on the final vector and is sensitive to the exact bit representation of its elements, differences may arise even when the numerical computation is considered correct. The Rayleigh-like value is therefore used as the main numerical validation criterion.

3 Results

In this section, the results of each experiment performed on the different versions are shown and commented. The experiments have been executed on nodes of the partition `normal` of the cluster.

The set of values tested for some parameters (e.g. chunk size, number of threads and number of MPI ranks) is a subset of the possible values for the parameters. This set has been selected a-priori, and is not exhaustive, to avoid overloading the cluster with experiments.

Results are averaged over 3 runs to minimize the impact of OS scheduling and MPI network congestion.

3.1 Strong Scaling

Strong scaling analysis is performed keeping the workload constant to $N = 500,000$, with $nnz = 20,000,000$, with both regular and irregular distribution of the nnzs. The ideal execution time is therefore inversely proportional to the number of workers:

$$T(p) = \frac{T(1)}{p}.$$

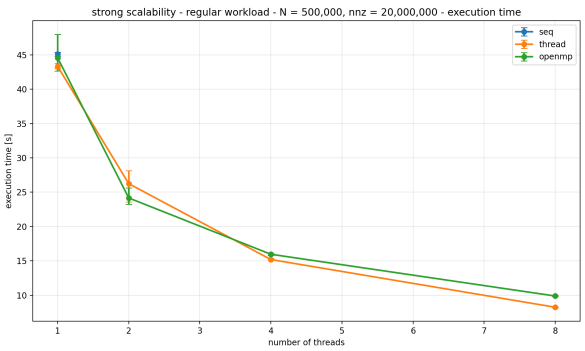

Figure 1: Strong scaling with regular workload

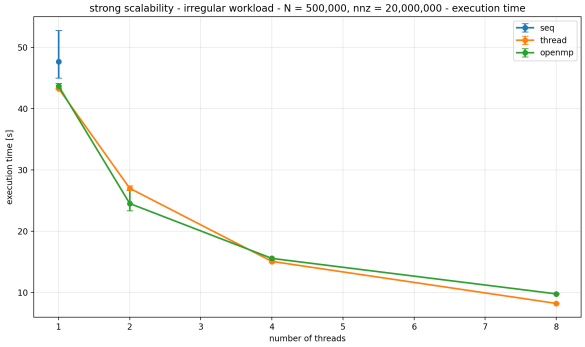

Figure 2: Strong scaling with irregular workload

As we can see in Figure 3.1, the strong scaling experiment yields almost the same results for both the regular and irregular workloads for the Thread and task-based OpenMP versions. This is likely because of the dynamic scheduling implemented in both the versions.

It's interesting to notice that, when using `t < 4` workers, OpenMP and Thread are pretty much equivalent, so their overhead in the thread creation and management (with the shared queue of tasks) is almost the same. When using `t = 8` threads, the Thread version is a couple of seconds faster than the OpenMP version (8s vs 10s).

Also, the scaling is not linear: when going from `t = 1` to `t = 2` the scaling is almost 1.7× for the Thread version and 1.8× for the OpenMP. From `t = 2` to `t = 4`, it is respectively 1.7× and 1.6×. From `t = 4` to `t = 8`, it is 1.9× and 1.5×.

In Figure 3.1, we can see that increasing the number of MPI ranks yields better results for both the regular and irregular workloads, with almost no difference in execution time between them. Doubling the number of ranks from 1 to 2 results in almost 1.67× speedup, and going from 2 to 4 ranks yields a 1.24× speedup, so the gain is far from the linear ideal speedup. That is because of the communication cost of MPI.

In terms of parallel efficiency (speedup divided by the number of workers), the Thread and OpenMP versions drop from approximately 90% efficiency at 2 threads to $60 - 68\%$ at 8 threads, confirming that the overhead from synchronization and reduction steps grows relative

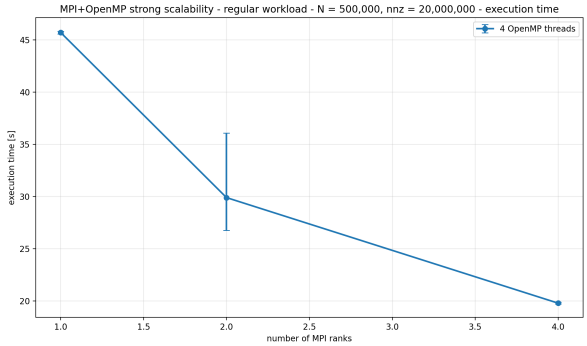

Figure 3: Strong scaling with regular workload (MPI+OpenMP)

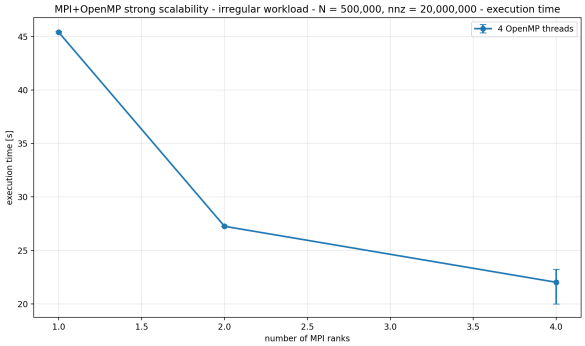

Figure 4: Strong scaling with irregular workload (MPI+OpenMP)

to the useful work as parallelism increases. The MPI+OpenMP version shows a similar trend at a much smaller scale, dropping from perfect efficiency at 1 rank to roughly $50 - 55\%$ efficiency at 4 ranks, dominated by the communication cost discussed in the MPI Phase Breakdown.

This behaviour is expected for a memory-bound sparse computation, where increasing the number of threads does not necessarily provide a proportional increase in memory bandwidth.

3.2 Weak Scaling

Weak scaling analysis is performed keeping the workload proportionate to the number of workers, going from $N = 100,000$, with $nnz = 4,000,000$ in case of `t = 1`, and the scaling up with `t`.

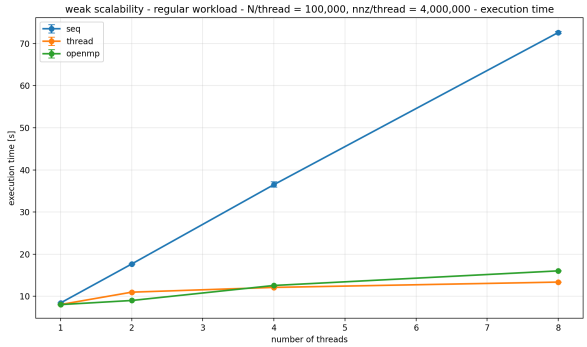

Figure 5: Weak scaling with regular workload

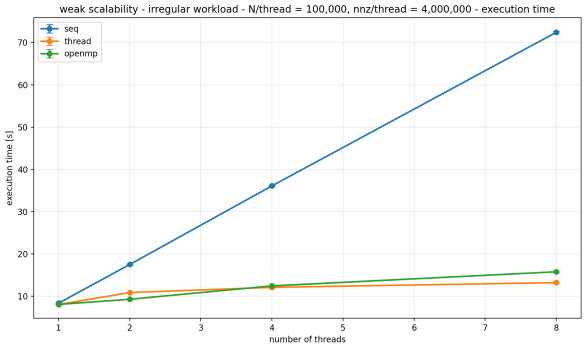

Figure 6: Weak scaling with irregular workload

Looking at Figure 3.2, as we would expect, the sequential program's execution time grows linearly with respect to the dimension of the problem. We also would expect to have a constant time for the Thread version and for the OpenMP version as the dimension of the dataset and the number of workers grow, but we go from 8 seconds in the case of `t = 1` to 13s for the Thread version and 15.7s for the OpenMP version. This could mainly be because of the overhead added by the synchronization between threads and the sequential parts of computation (e.g. reduction steps from local partial results). Again, the regular and irregular workloads have the same execution time.

In Figure 3.2, we can see that the execution time of the MPI version is clearly far from the horizontal ideal line that we would like to obtain in a perfectly parallel implementation. That is because of the synchronization between OpenMP workers inside the single rank and communication between different ranks (which is much more heavy in terms of execution time). In fact, we get that, going from 1 rank with 4 OpenMP threads to 2 ranks with 4 OpenMP threads each, we get an execution time of almost $1.4\times$ for the regular workload and $1.5\times$ for the irregular workload, and going from 2 to 4 ranks we get respectively $1.4\times$ and $1.2\times$.

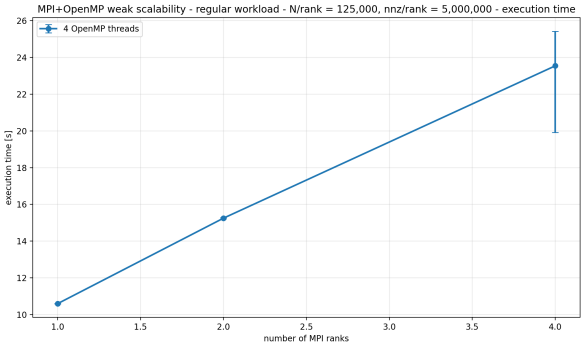

Figure 7: Weak scaling with regular workload (MPI+OpenMP)

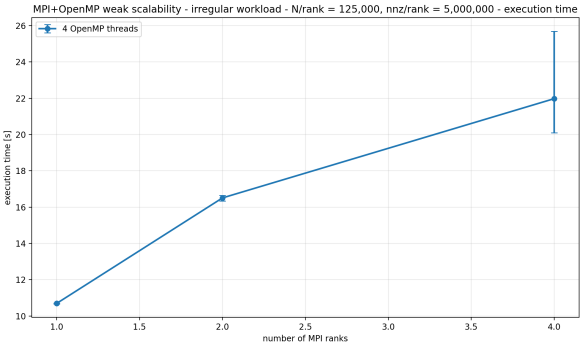

Figure 8: Weak scaling with irregular workload (MPI+OpenMP)

3.3 Speedup

Speedup analysis is performed computing the following formula:

$$\text{Speedup}(n) = \frac{T(1)}{T(n)}$$

where $T(1)$ is the execution time for a single thread and $T(n)$ is the execution time using n threads. For the MPI experiment, the number of OpenMP threads is kept constant and the number of ranks is incremented.

The analysis is performed keeping the workload constant to $N = 500,000$, with $nnz = 20,000,000$, with both regular and irregular distribution of the nnzs.

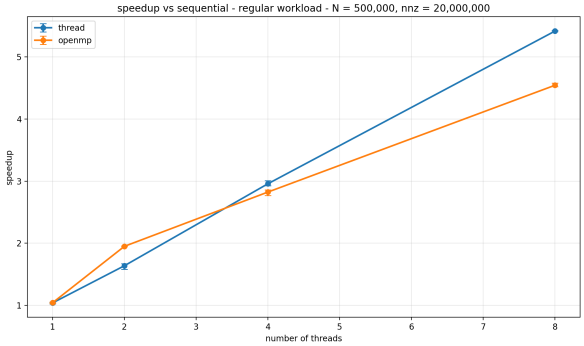

Figure 9: Speedup with regular workload

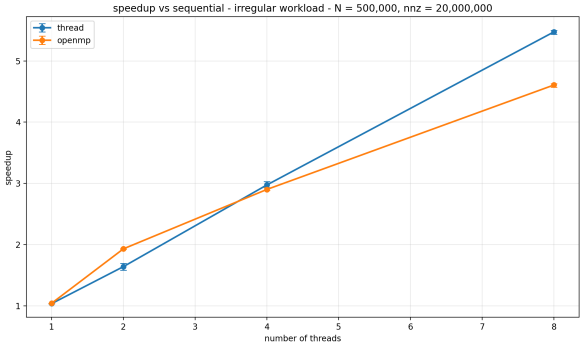

Figure 10: Speedup with irregular workload

The ideal speedup would be of n. Looking at Figure 3.3, we can see that it is not linear, and that might be because of the overhead in the creation and management of the threads and tasks. For `t = 2`, OpenMP is clearly the best choice with an almost linear speedup, but as the number of workers grows, the Thread version has a higher speedup. Just like in the previous experiments, the irregularity of the matrix does not affect the performances of the executables.

For the MPI version in Figure 3.3, the speedup is almost the same for both the workloads, considering the whiskers in the plots. As seen in strong scaling experiment (Figure 3.1), the speedup is non linear, and that is mainly because of the communication overhead in MPI, but also because of the synchronization overhead of OpenMP.

3.4 Thread scheduling effect

In this experiment, the Thread version has been executed on the cluster keeping the workload constant to $N = 500,000$, with $nnz = 20,000,000$, with both regular and irregular distribution

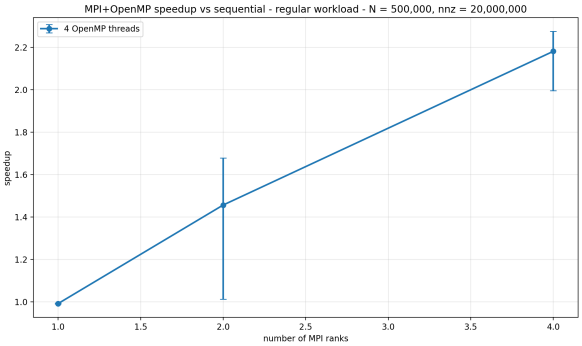

Figure 11: Speedup with regular workload (MPI+OpenMP)

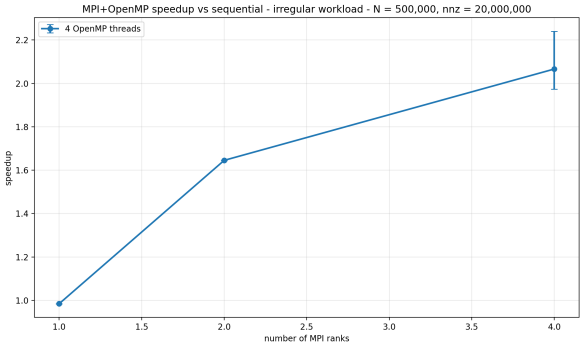

Figure 12: Speedup with irregular workload (MPI+OpenMP)

of the nnzs, and sweeping the scheduling technique (`static` vs `dynamic`) and chunk size (number of rows to assign to each task).

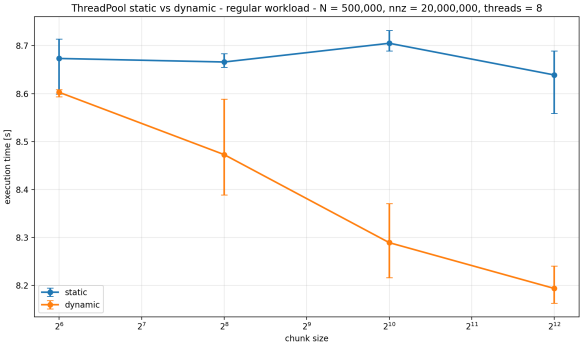

Figure 13: Thread scheduling analysis with regular workload

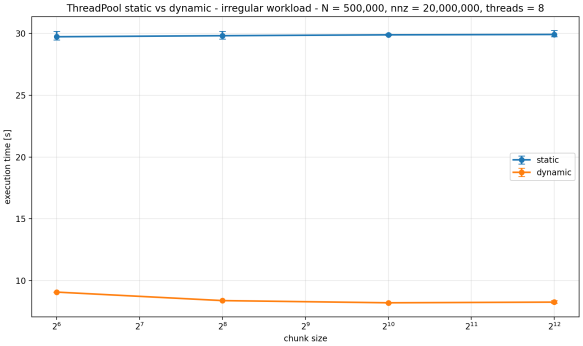

Figure 14: Thread scheduling analysis with irregular workload

We can observe in Figure 3.4 that, when using the regular workload, execution time is quite the same for static and dynamic scheduling. It actually does make sense, since all the rows have the same number of non zero elements, but the dynamic scheduling has slightly lower execution time.

In the irregular matrix, the choice is obvious: the dynamic scheduling is mandatory for this type of task, since some rows are denser than other, and, in the worst case for the static scheduling, we might result in a situation where a single worker gets majority of the work and all the other workers sleep until its completion. The dynamic approach resolves this problem with any of the chunk sizes used in the experiment. It's interesting to notice that, for small chunk sizes (`chunk_size = 64`) the execution time is higher than the one for big chunk sizes (`chunk_size = 4096`). This could be due to the overhead in the creation of the tasks and the management of the task queue, and the smaller granularity does not compensate for that overhead.

3.5 OpenMP granularity sweep

In this experiment, OpenMP task-based has been tested with fixed number of threads `t = 8` and with different chunk sizes to define the tasks.

Independently from the workload distribution, we can see in Figure 3.5 that using 4096 rows as task dimension yields the lowest execution time. This is probably due to the fact that using lower task size results in more overhead in synchronization between threads and higher task

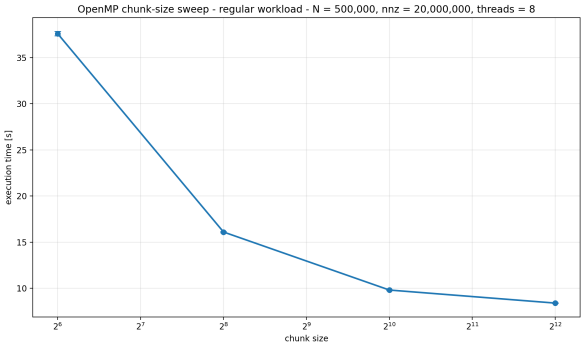

Figure 15: OpenMP granularity sweep with regular workload

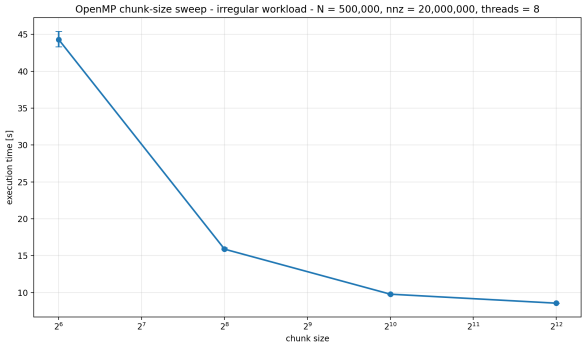

Figure 16: OpenMP granularity sweep with irregular workload

creation time. At some point, using a larger value of chunk size will result in a spike in the execution time, but this value is not in the range tested (`[64, 256, 1024, 4096]`).

3.6 OpenMP task-based vs OpenMP worksharing

In this experiment, the OpenMP task-based version has been compared to the worksharing version, and both have been executed on the cluster keeping the workload constant to $N = 500,000$, with $nnz = 20,000,000$, with both regular and irregular distribution of the nnzs, and sweeping the number of threads `t = [1, 2, 4]`.

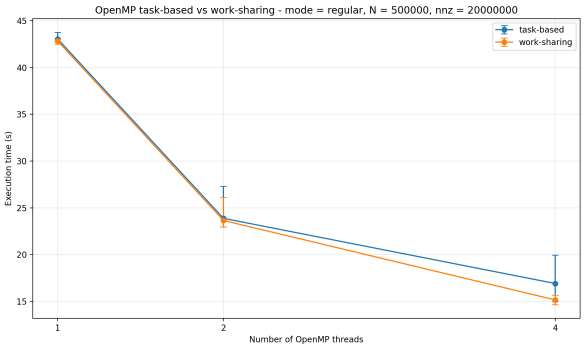

Figure 17: OpenMP task-based vs worksharing analysis with regular workload

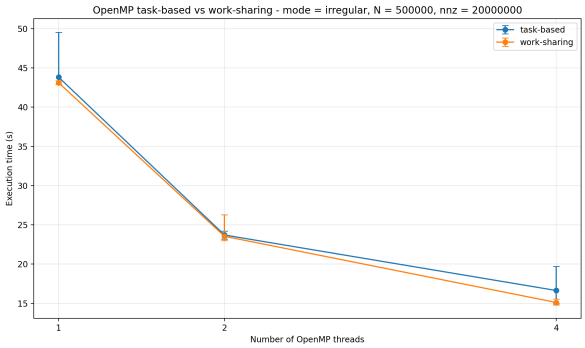

Figure 18: OpenMP task-based vs worksharing analysis with irregular workload

In Figure 3.6, we can see that the impact of using a regular workload compared to an irregular workload is negligible, and using the worksharing version results in lower execution time by a couple of seconds with respect to the task-based version when using multiple threads. This behaviour is predictable, since the overhead for creating `num_chunks = n / chunk_size` tasks in the task-based version is higher than scheduling tasks in the worksharing version, which lets the OpenMP library manage the chunks of data to work on internally, and assigning them with the dynamic scheduling to the workers. The task-based version is also more expensive because of the `#pragma omp single` segment, which only lets a single thread execute the code, and create the tasks to add to the task queue in a sequential way. The worksharing version instead lets all the threads run in parallel the code inside the `#pragma omp parallel num_threads(num_threads)` segment, so the `single` directive bottleneck is avoided.

3.7 MPI+OMP Phase Breakdown

In this experiment, MPI+OpenMP has been tested sweeping on `n = [1, 2, 4]` MPI ranks and `t = [2, 4]` OpenMP workers on a fixed workload constant to $N = 500,000$, with $nnz = 20,000,000$, with both regular and irregular distribution of the nnzs.

The MPI+OpenMP execution time is decomposed into these main components:

1. **`global_spmv`**: SpMV kernel execution using OpenMP threads inside each single MPI rank,
2. **`global_epoch`**: epoch transition time, often negligible (in the order of 10^{-6}),
3. **`global_comm`**: communication cost between different MPI ranks,
4. **`global_rotate`**: time spent rotating the **y** result vector at each epoch,
5. **`global_normalize`**: normalization operation cost.

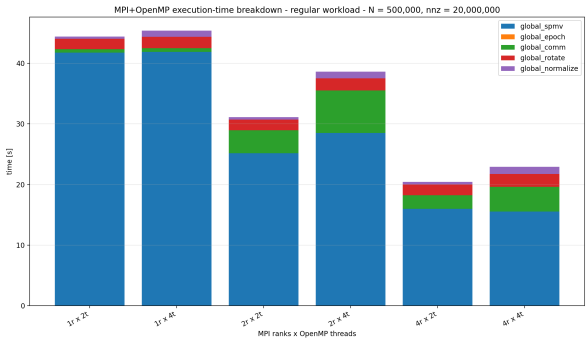

Figure 19: MPI+OpenMP phases time breakdown with regular workload

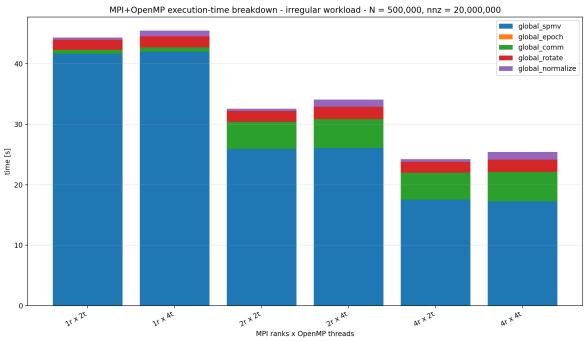

Figure 20: MPI+OpenMP phases time breakdown with irregular workload

Looking at Figure 3.7, we can see that using 4 OpenMP workers instead of 2 always increases the execution time, in particular for the `normalize` phase (which doubles when going from 2 threads to 4 threads, because of the overhead in local synchronization with relatively small **y** vector), but also for the communication phase, which is counterintoutive. A possible factor that causes the observed communication overhead with multiple OpenMP threads is the Non-Uniform Memory Access (NUMA) architecture of the underlying hardware. The compute nodes (`node01, node02`) utilized for the experiments are dual-socket systems (Intel Xeon E5-2640 v2) presenting two distinct NUMA nodes. As the number of OpenMP threads per MPI rank increases, the operating system inevitably schedules worker threads across both physical sockets to distribute the computational load. Consequently, portions of the `local_out` array computed by worker threads are physically allocated and written to remote memory banks. Because the MPI communication is funneled through the master thread, the packing of data for the `MPI_Allgatherv` collective forces the master to fetch data across the inter-socket link. This NUMA effect introduces severe memory latencies in communication time. A possible follow-up experiment would control OpenMP thread affinity and compare single-socket and dual-socket executions in order to determine whether NUMA effects are responsible for the observed increase.

However, increasing the number of MPI ranks yields lower execution time, as expected.

3.8 MPI+OMP rank and threads sweep

In this experiment, MPI+OpenMP has been tested sweeping on `n = [1, 2, 4]` MPI ranks and `t = [2, 4]` OpenMP workers on a fixed workload constant to $N = 500,000$, with $nnz = 20,000,000$, with both regular and irregular distribution of the nnzs.

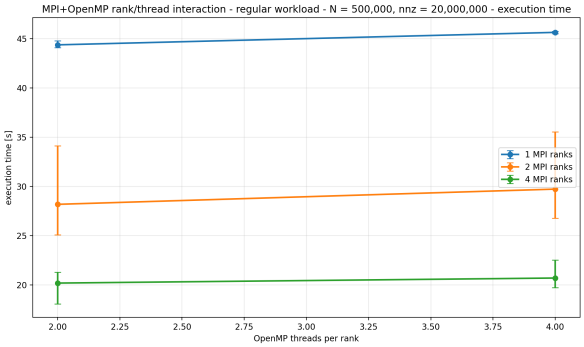

Figure 21: MPI+OpenMP rank and thread sweep with regular workload

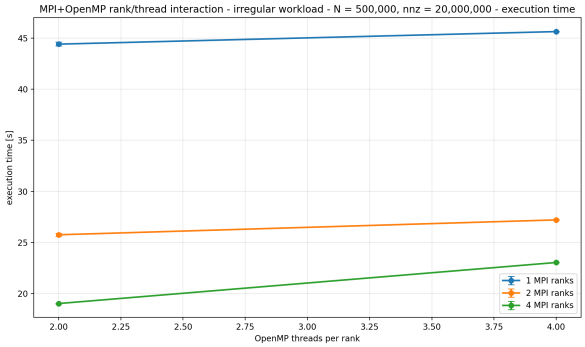

Figure 22: MPI+OpenMP rank and thread sweep with irregular workload

Looking at Figure 3.8, it's clear that using more ranks results in lower execution time, as seen in Figure 3.7, but not in a linear trend because of the communication between ranks. Also, adding more threads yields worse results, with higher execution time, as state in the previous experiment.

It's also interesting to note that the whiskers in the plot increase using 2 and 4 ranks, because of the network communication part, which highly depends on the network congestion. Instead, when using a single rank, the whiskers are almost flat, since no communication happens.

4 Conclusions

This work implemented and analyzed four parallel formulations of an iterative sparse matrix-vector computation on evolving sparse matrices: a custom ThreadPool, two OpenMP variants (task-based and work-sharing), and a hybrid MPI+OpenMP design. All implementations were validated against the sequential reference through a weak correctness check on the Rayleigh value, within a tolerance of 10^{-9}, confirming that the parallel results are numerically consistent with the sequential baseline despite the different floating-point operations' order introduced by parallelization.

Across all shared-memory experiments, dynamic scheduling proved essential for the irregular workload, preventing the worst-case scenario where a single worker is assigned a disproportionate share of dense rows while the others remain idle. Between the two OpenMP variants, the work-sharing version consistently outperformed the task-based one, since the latter incurs both the per-task creation overhead and a serial task-generation phase carried out by a single thread before the remaining workers can start executing.

The MPI+OpenMP version showed sub-linear strong and weak scaling, dominated by communication cost between ranks. A counterintuitive increase in communication time was observed when increasing the number of OpenMP threads per rank, tentatively attributed to NUMA effects on the dual-socket cluster nodes.

The experiments reveal different scalability bottlenecks depending on the execution model. In the shared-memory implementations, scalability is mainly limited by thread synchronization, task scheduling overhead, and the memory-bandwidth requirements of the SpMV operation. The effect of load imbalance becomes more important for irregular matrices, making dynamic scheduling preferable.

In the MPI+OpenMP implementation, the dominant bottleneck is the communication between ranks. Increasing the number of MPI ranks reduces the amount of local computation, but the cost of collective communication becomes increasingly significant. Similarly, increasing the number of OpenMP threads within a rank does not necessarily improve performance .

The adopted row-based partitioning introduces an additional potential source of imbalance.

Although the partitioning attempts to balance the number of non-zero elements, complete balance cannot always be achieved because rows cannot be arbitrarily split without complicating the synchronization and communication phases.

4.1 Limitations

The row-partitioning strategy described in the MPI+OMP version, while being a reasonable compromise between simplicity in implementation and load balancing, can still assign a disproportionate amount of nonzeros to the last rank under highly skewed distributions. A per-element partitioning scheme would guarantee a more fair load balancing at the cost of significantly increased implementation and communication complexity, since multiple ranks could then hold nonzeros belonging to the same row.

Also, executing other experiments for the NUMA hypothesis could help pinpointing the exact cause of the increase in execution time when keeping the number of ranks fixed and incrementing the number of OpenMP workers.